30-Day FreeRTOS Course for ESP32 Using ESP-IDF

(Day 20)

"Interrupt Management in FreeRTOS"

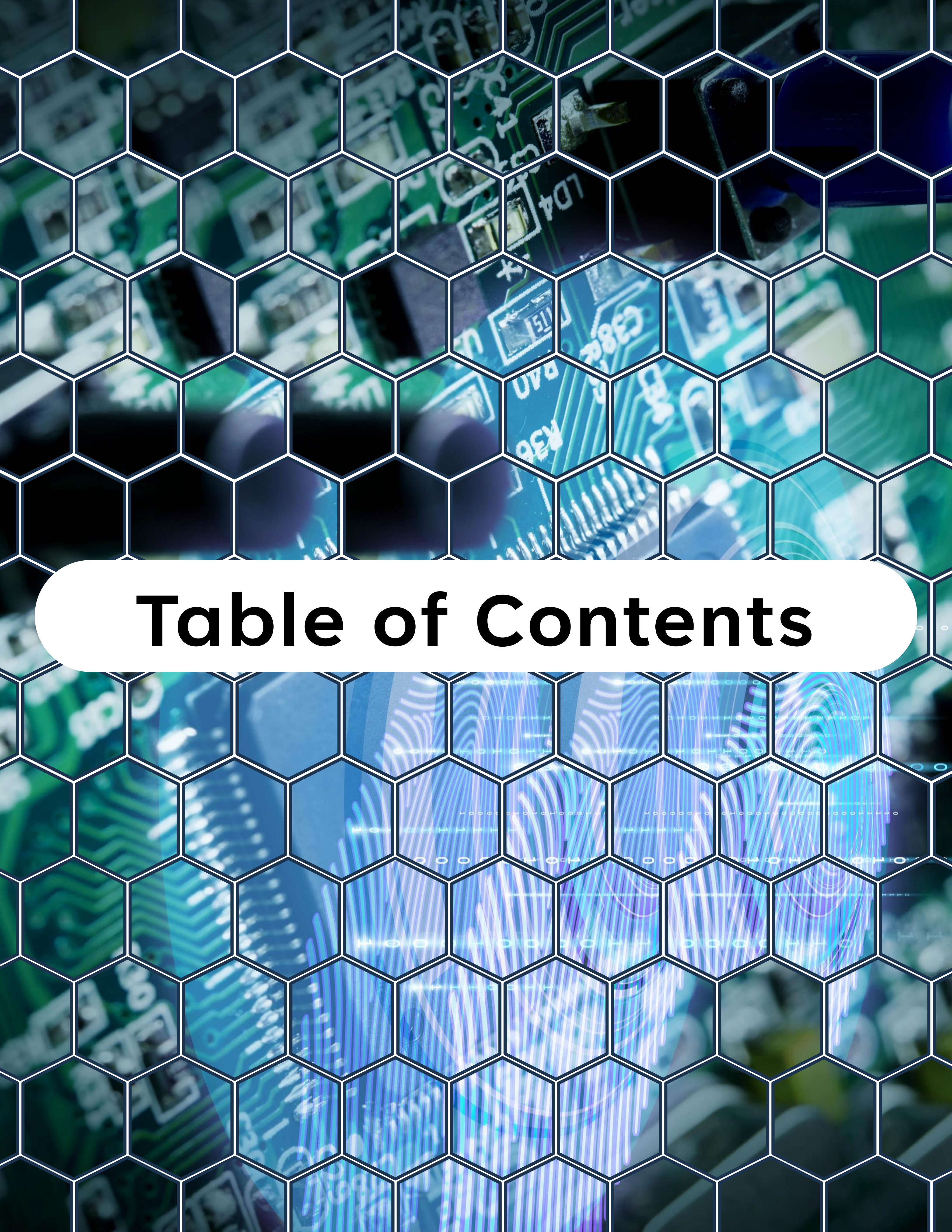
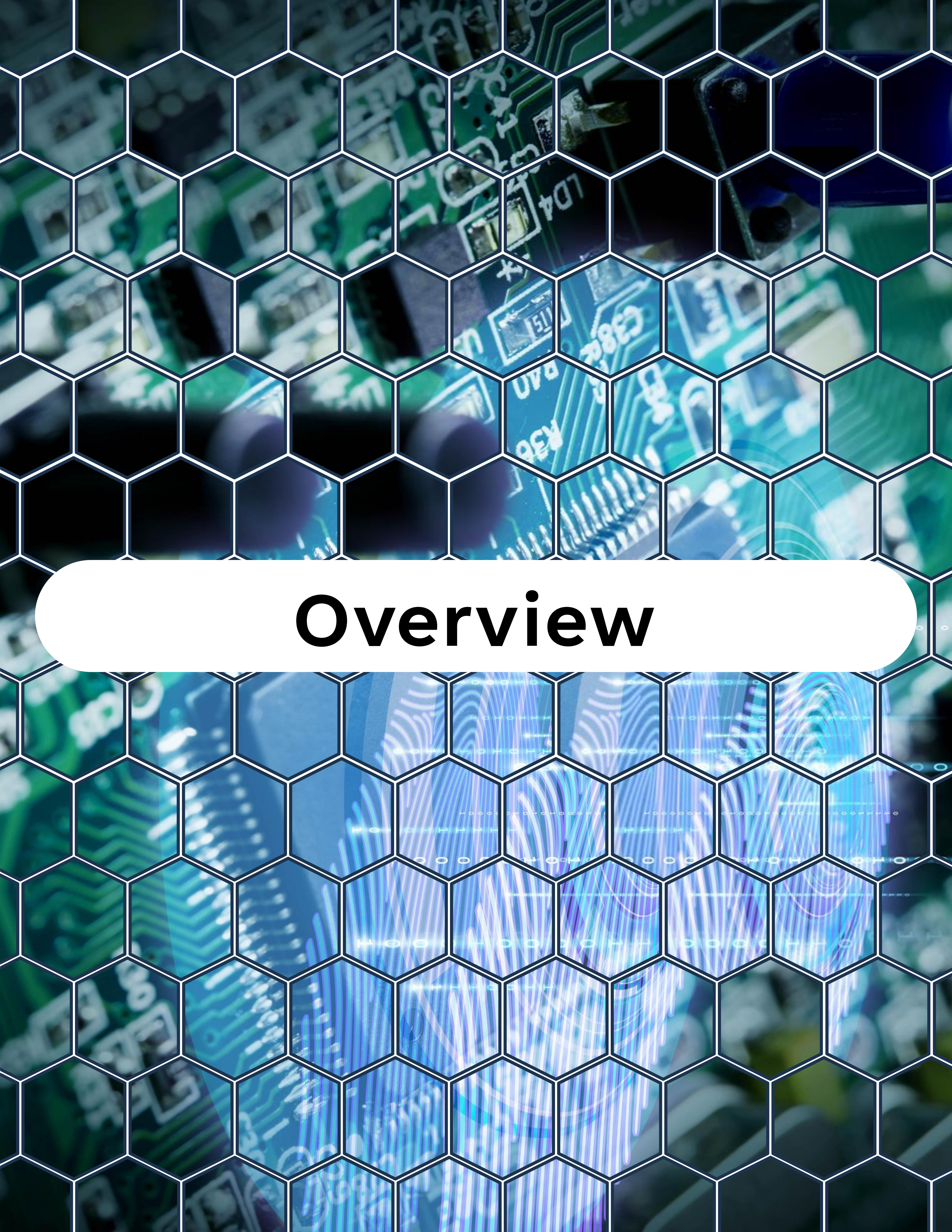


Table of Contents

Table of Contents

Overview



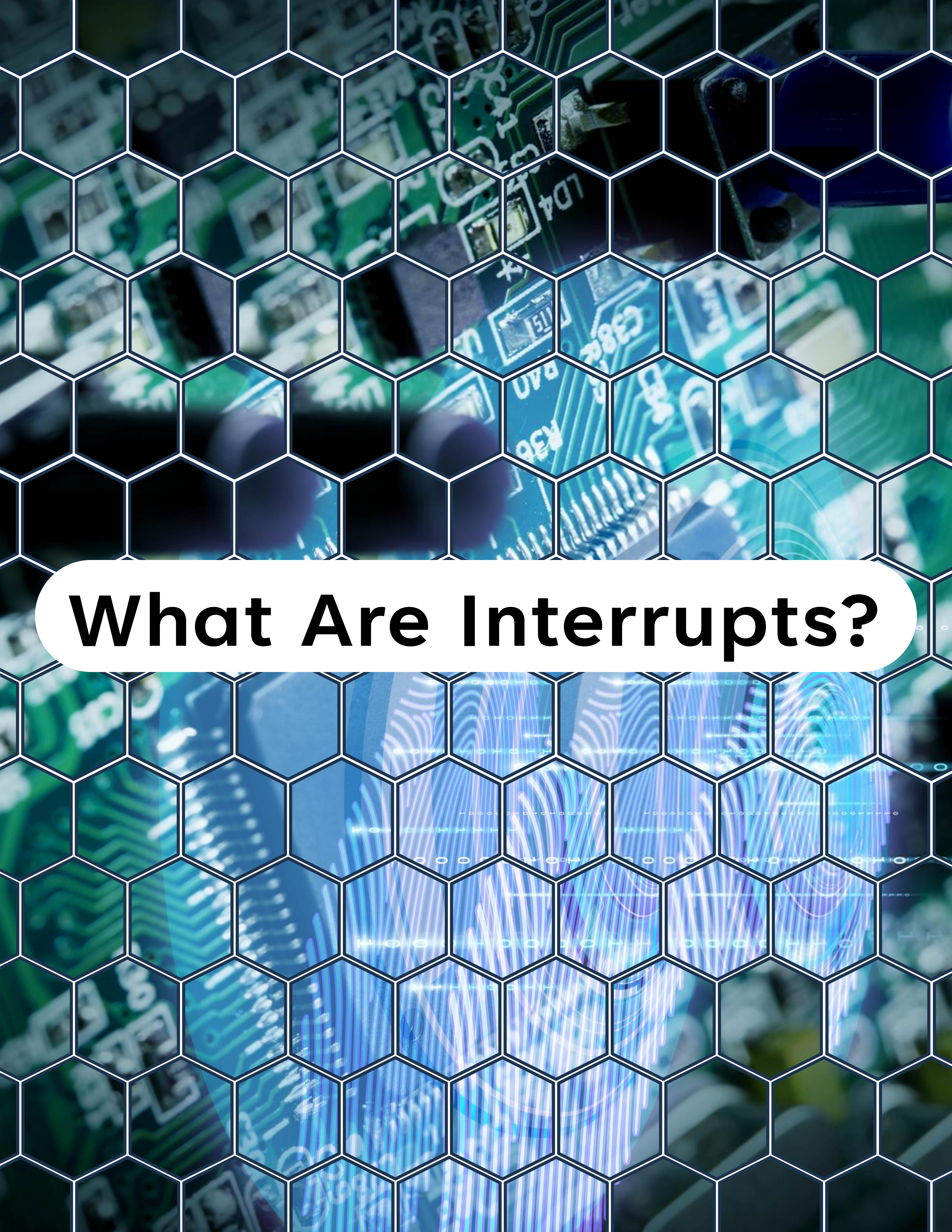
Overview

On **Day 20**, you'll learn:

- How interrupts work in ESP32 and FreeRTOS
- The rules for writing **safe Interrupt Service Routines (ISRs)**
- How ISRs interact with FreeRTOS primitives (queues, semaphores, task notifications)
- The difference between **normal functions and ISR-safe functions**
- A practical example: **Button press ISR → Task handling logic**
- Best practices for designing responsive, efficient ISRs

By the end of this lesson, you'll understand how to safely use interrupts in FreeRTOS applications on ESP32.

What Are Interrupts?



What Are Interrupts?

An interrupt is a hardware signal that tells the CPU to pause normal execution and immediately run a special function called an ISR (Interrupt Service Routine).

Examples:

- GPIO pin change (button press)
- UART receiving a byte
- Timer expiry
- ADC conversion complete

📌 ISRs must be **fast, short, and non-blocking**. Any heavy processing should be **deferred to a FreeRTOS task**.

ISRs and FreeRTOS

ISRs and FreeRTOS

FreeRTOS allows ISRs to interact with tasks, but only via ISR-safe APIs.

General Rules for ISRs in FreeRTOS:

- Never call **blocking functions** inside ISRs (e.g., `vTaskDelay()`).
- Use only **FromISR** versions of FreeRTOS functions.
- If an ISR wakes a higher-priority task, call `portYIELD_FROM_ISR()`.

ISR-Safe Functions

ISR-Safe Functions

Examples of ISR-safe APIs in FreeRTOS:

Queues:

- xQueueSendFromISR()
- xQueueReceiveFromISR()

Semaphores:

- xSemaphoreGiveFromISR()

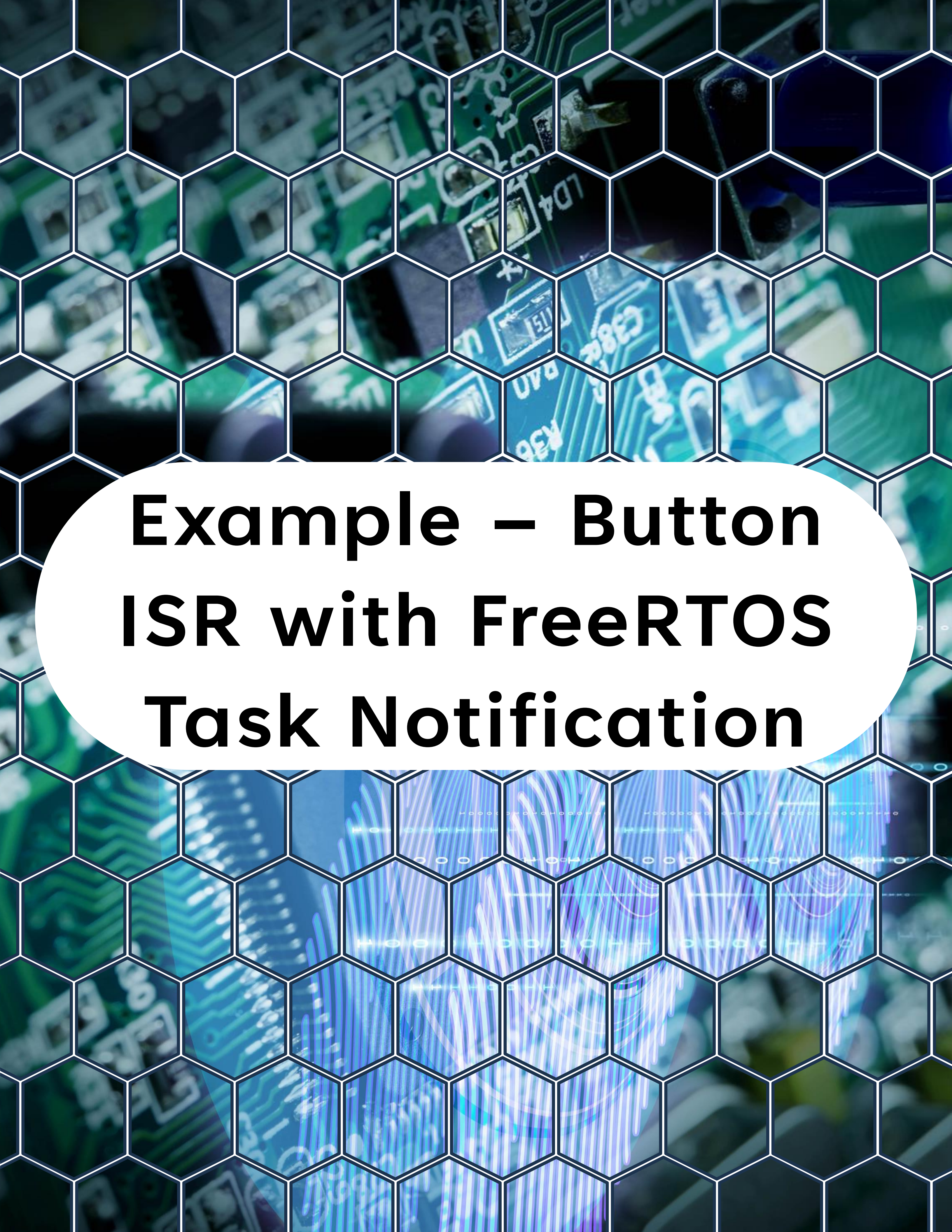
Event Groups:

- xEventGroupSetBitsFromISR()

Task Notifications:

- xTaskNotifyFromISR()
- vTaskNotifyGiveFromISR()

📌 Normal versions (xQueueSend, xSemaphoreGive, etc.) must not be used in ISRs.



Example – Button ISR with FreeRTOS Task Notification

Example – Button ISR with FreeRTOS Task Notification

Code Example

```c
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"
#include "esp_log.h"

#define BUTTON_GPIO 0
#define TAG "DAY20"

// Task handle for the button task
TaskHandle_t button_task_handle = NULL;

// ISR handler
static void IRAM_ATTR button_isr_handler(void *arg) {
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;

    // Notify the task that a button press occurred
    vTaskNotifyGiveFromISR(button_task_handle,
                           &xHigherPriorityTaskWoken);

    // Force context switch if needed
    if (xHigherPriorityTaskWoken) {
        portYIELD_FROM_ISR();
    }
}

// Task handling the button press
void button_task(void *pvParameter) {
    while (1) {
        // Wait indefinitely until ISR notifies
```

Example – Button ISR with FreeRTOS Task Notification

```c
// Task handling the button press
void button_task(void *pvParameter) {
    while (1) {
        // Wait indefinitely until ISR notifies
        ulTaskNotifyTake(pdTRUE, portMAX_DELAY);
        ESP_LOGI(TAG, "Button pressed, handled in task!");
    }
}

// Main application entry point
void app_main() {
    // Configure button pin
    gpio_config_t io_conf = {
        .intr_type = GPIO_INTR_NEGEDGE,
        .mode = GPIO_MODE_INPUT,
        .pin_bit_mask = (1ULL << BUTTON_GPIO),
        .pull_up_en = GPIO_PULLUP_ENABLE,
        .pull_down_en = GPIO_PULLDOWN_DISABLE
    };
    gpio_config(&io_conf);

    // Create task
    xTaskCreate(button_task, "ButtonTask", 2048, NULL, 10,
                &button_task_handle);

    // Install ISR service
    gpio_install_isr_service(0);

    // Attach the ISR handler for the button GPIO
    gpio_isr_handler_add(BUTTON_GPIO, button_isr_handler, NULL);
}
```

Example – Button ISR with FreeRTOS Task Notification

Expected Behavior

When you press the button:

- ISR runs, sends a notification to the task.
- The task unblocks and logs:

```
I (1234) DAY20: Button pressed, handled in task!
```

- ISR remains lightweight (only signaling), while the task does the actual work.

Best Practices for ISRs in FreeRTOS

Best Practices for ISRs in FreeRTOS

✅ **Keep ISRs as short as possible** — only signal, don't process.

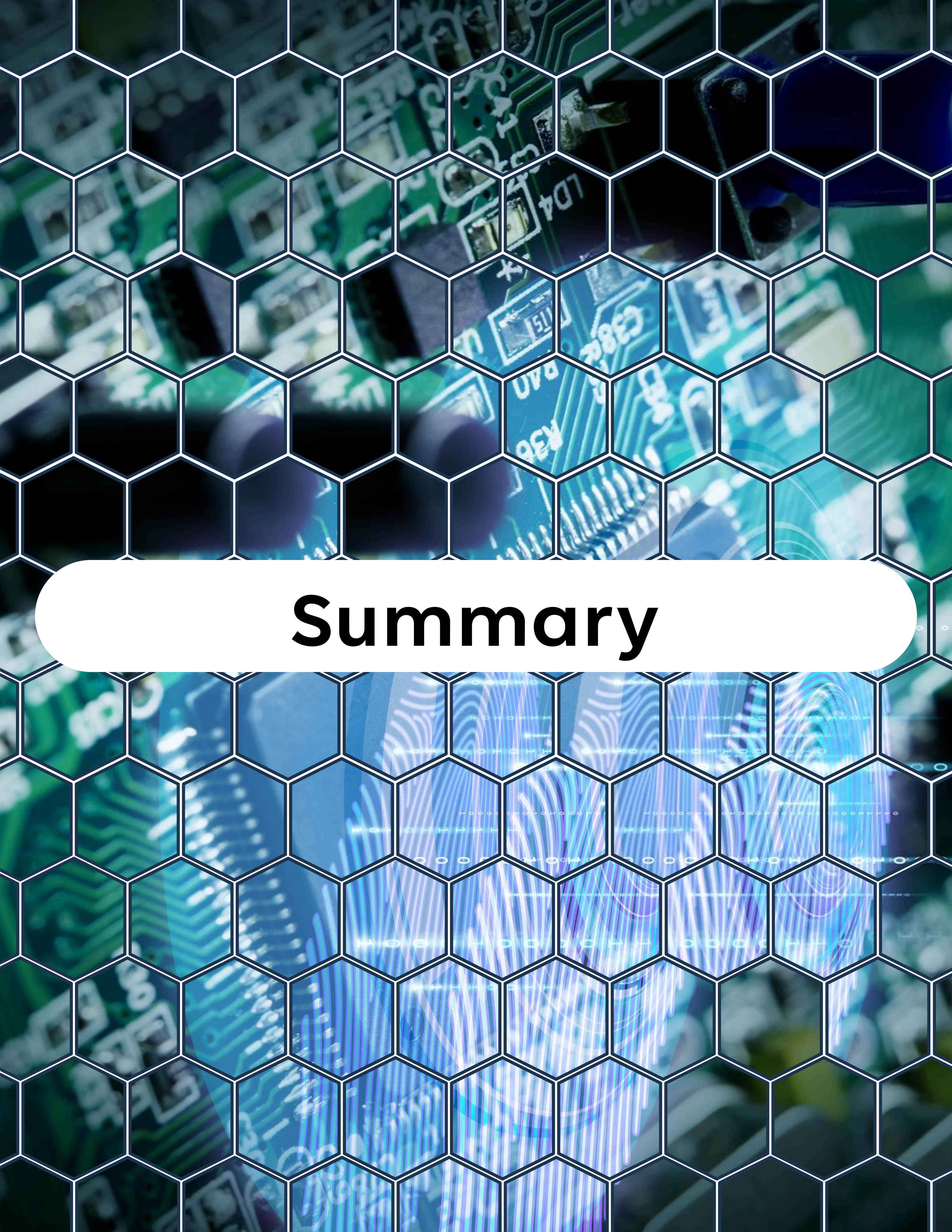
✅ Always use **FromISR versions** of FreeRTOS functions.

✅ Use `portYIELD_FROM_ISR()` if a higher-priority task was woken.

✅ Use **task notifications** for fastest signaling ISR → Task.

✅ Use **queues** or **message buffers** if data transfer is needed.

✅ Avoid using printf/logging directly in ISR — do it in a task.

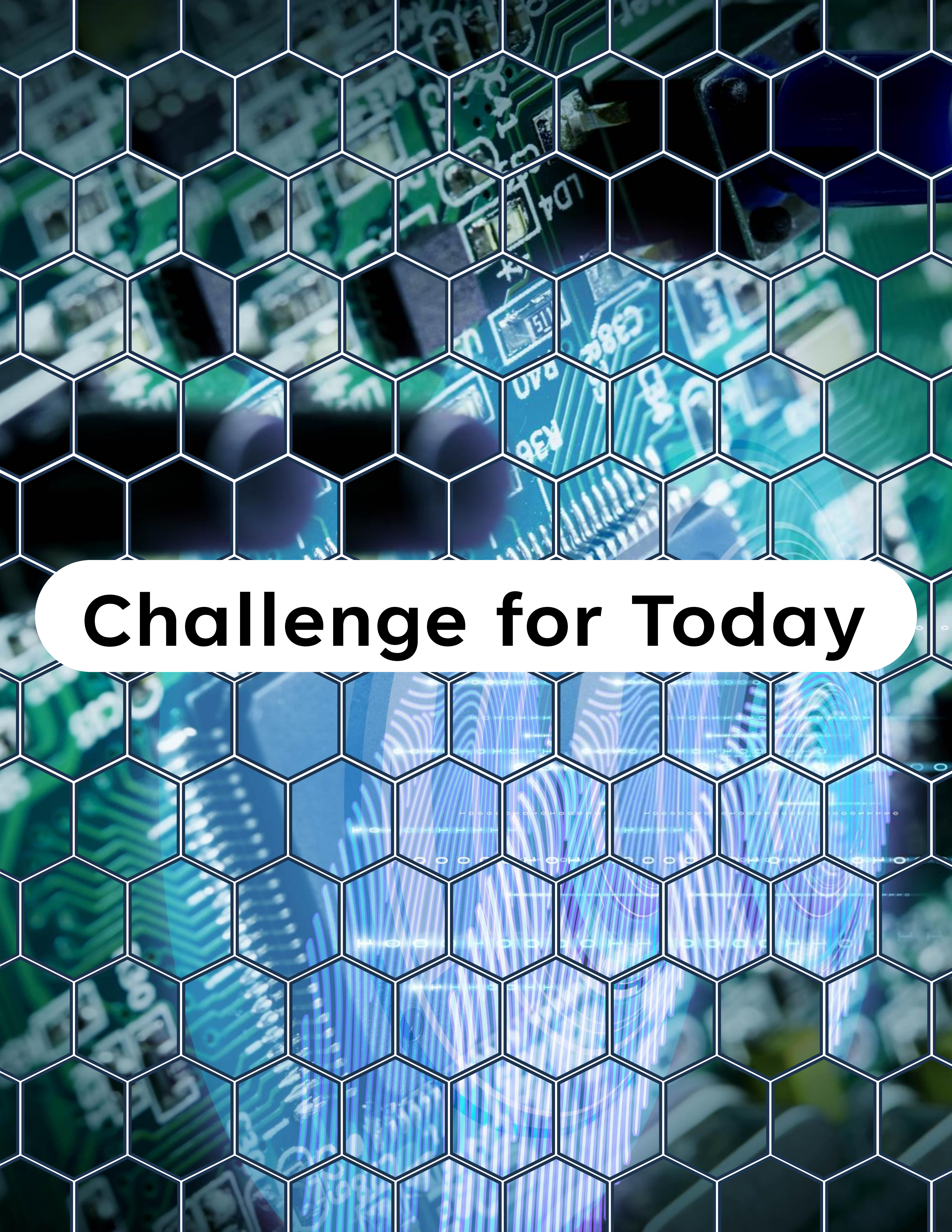


Summary

Summary

- Interrupts allow ESP32 to respond quickly to hardware events.
- ISRs must be **fast**, **non-blocking**, **and safe**.
- FreeRTOS provides **FromISR APIs** for signaling tasks.
- Heavy work should be deferred to tasks, not done inside ISRs.
- Best option: use **task notifications** for efficiency, or **queues/buffers** for data transfer.

Challenge for Today



Challenge for Today

🛠️ Extend today's project by:

1. Adding a **second button ISR** that sends its GPIO number via a **queue**.
2. Modify the task to handle both button presses differently (e.g., short press vs long press).
3. Compare performance between **queue-based signaling** and **task notifications**.

"Thanks for watching!

If you enjoyed this video,

make sure to hit the like button and subscribe to stay updated with my latest content.

Don't forget to check out my other videos for more tips and tutorials on Embedded C, Python, hardware designs, etc. Keep exploring, keep learning, and I'll see you in the next video!"

https://www.linkedin.com/in/yamil-garcia
https://www.youtube.com/@LearningByTutorials
https://github.com/god233012yamil